Class and Object in Python

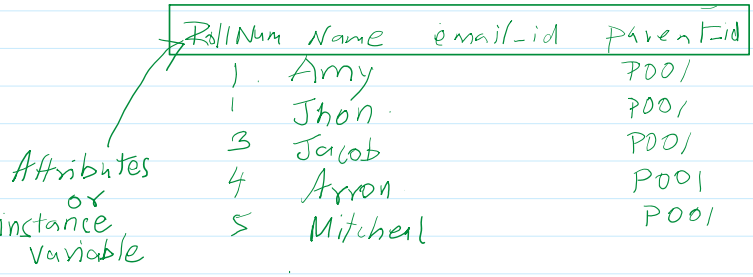
22 July 2026 06:51

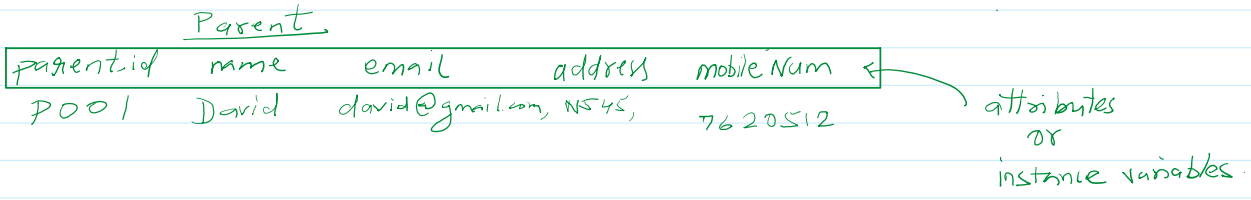
What a class and an object are(analogy)

- **Class: a blueprint, template or recipe that defines attributes(data) and behaviors(methods) for a type of thing.**

  Student(rollNum, Name, email_id, parent_id)

  Parents(parent_id, name, email, address, mobileNum)





- **Object(instance): a concrete item created from the blueprint-it holds its own attribute values and can use the class behaviors. Example analogy: class = blueprint of a car; object = one particular car**

Basic class definition and creating objects

- **Define a class using the class keyword.**
- **Create objects by calling the class like a function.**

is the constructor (runs when an instance is created). The first parameter of instance method is conventionally named self and refers to the instance.

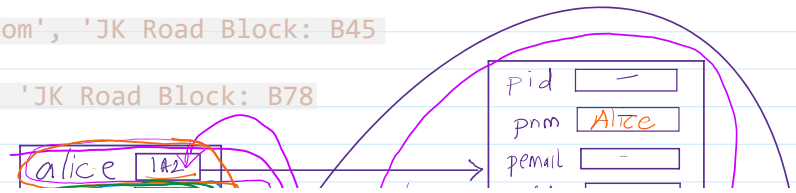
```
class Parent:
    def __init__(self, pid, pnm, pemail, paddr, pmobile):
        self.pid = pid
        self.pnm = pnm
        self.pemail = pemail
        self.paddr = paddr
        self.pmobile = pmobile

    def greet(self):
        return f"Hello {self.pnm} Good Afternoon"

# main code
alice = Parent('p101', 'Alice', 'alice@g.com', 'JK Road Block: B45
Texsus', '9876523')
bob = Parent('p102', 'Bob', 'alice@g.com', 'JK Road Block: B78
Texsus', '98765432')
```

Attributes assigned to self (self.var_name) are instance attributes — each object gets its own copy.

this invokes __init__(self, ...) method

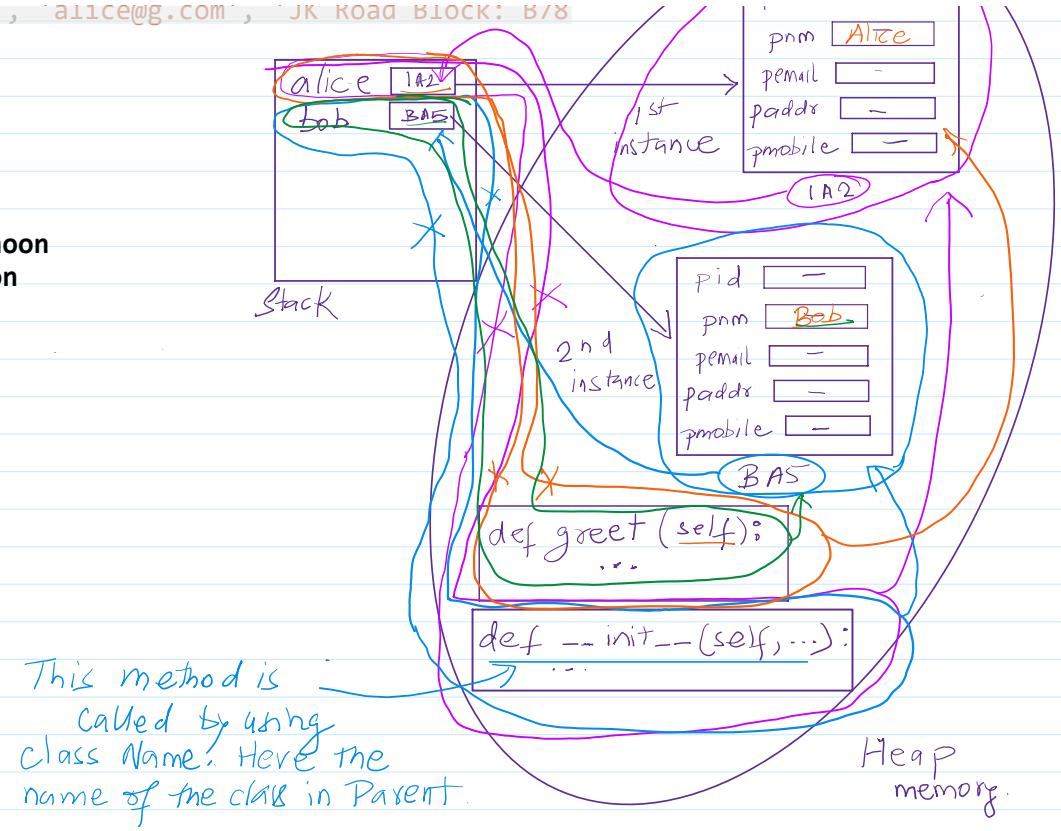


```
bob = Parent('p102', 'Bob', 'alice@g.com', 'JK Road Block: B78
Texsus', '98765432')

print(alice.greet())
print(bob.greet())
```

Output:

Hello Alice Good Afternoon
Hello Bob Good Afternoon



Homework

Programming Questions

1. **Student-Parent Management System Write a Python program to design two classes:**
   - **Student with attributes: rollNum, name, email_id, parent_id.**
   - **Parent with attributes: parent_id, name, email, address, mobileNum. Tasks:**
   - **Create multiple student and parent objects.**
   - **Implement a method to link each student with their parent using parent_id.**
   - **Display a report showing each student's details along with their parent's contact information.**
   - **Add functionality to update a parent's mobile number and reflect the change in the student report.**

2. **Car Dealership Inventory System Create a class Car with attributes: brand, model, price, year. Tasks:**
   - **Write methods to calculate depreciation of the car's value (e.g., 10% per year).**
   - **Create multiple car objects and store them in a list.**
   - **Implement a search function to find cars by brand or year.**
   - **Add functionality to display the most expensive car in the inventory.**
   - **Extend the program by creating a Customer class that can "buy" a car, reducing the dealership's inventory.**

3. **Library Management System Define two classes:**
   - **Book with attributes: book_id, title, author, available_copies.**
   - **Member with attributes: member_id, name, email. Tasks:**
   - **Allow members to borrow books (decreasing available_copies).**
   - **Prevent borrowing if no copies are available.**
   - **Implement a method to return books (increasing available_copies).**
   - **Display a report of all borrowed books along with the member details.**

- Add functionality to track overdue books (assume a fixed borrowing period of 14 days).

These problems will push you to think about object relationships, encapsulation, and methods in Python. They're designed to mimic real-world systems, so solving them will give you strong practice in applying OOP concepts.